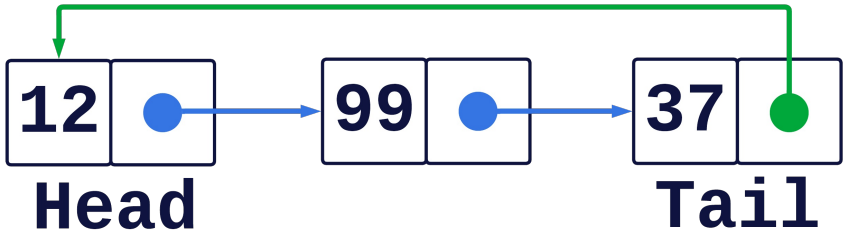
Learning Objectives

Learners will be able to...

- **Define a circular linked list**
- **Differentiate circular linked lists from singly linked and doubly linked lists**
- **Implement a circular linked list and perform insertion, deletion, and traversal/search**
- **Analyze the complexities of different linked list operations**

Circular Linked Lists

In contrast to singly and doubly linked lists where the last node points to `nullptr`, a circular linked list is slightly different. In a **circular linked list (CLL)**, the last node points back to the first node, thereby forming a loop. This unique characteristic has its applications and complexities, and we will delve into the specific operations that can be performed on a CLL. Circular linked lists offer unique advantages such as the ease of rotation, but they also come with their challenges, especially in terms of traversal and maintenance of the circular structure.



Implementation

Circular linked lists can be either singly or doubly linked. However, it is more common to see a circular singly linked list due to its lower memory overhead (one pointer vs two pointers). The big risk you run with a circular linked list is getting stuck in an infinite loop when traversing. To optimize performance, we are only going to manage the head and tail nodes.

```
class Node {
public:
    Node* next;
    int data;

    Node(int data) : data(data), next(nullptr) {}
};
```

Create the `CircularList` class that has the `head` and `tail` attributes. We are also going to create the `printList` method so we can see how operations alter the list. We are going to start the traversal the same way as singly and doubly linked lists. However, the defining characteristic of a circular list is that the tail points back to the head. That means there are no nodes with a pointer whose value is `nullptr`. Basically, we have initiated an infinite loop.

The logic behind traversing a circular list is to keep traversing until you get back to the head node. Because of this, the order of our code is important. The first thing you should do inside the while loop is the desired action. In

this case, we are printing a value (you could just as easily search for a value, modify a value, etc.). After the action is performed, then advance to the next node. Finally, ask if the new node is the head. If so, break out of the loop.

```
class CircularList {
public:
    Node* head;
    Node* tail;

    CircularList() : head(nullptr), tail(nullptr) {}

    void printList() {
        Node* current = head;

        while (current != nullptr) {
            std::cout << current->data << " ";
            current = current->next;

            if (current == head) {
                std::cout << std::endl;
                break;
            }
        }
    }
};
```

If we are keeping track of the tail, why not traverse until we reach the tail? Assume that we were to change the `printList` method to the code below. **Important**, this code is faulty, do not add it to your program.

```
    void printList() {
        Node* current = head;

        while (current != tail) {
            std::cout << current->data << " ";
            current = current->next;

            if (current == head) {
                std::cout << std::endl;
                break;
            }
        }
    }
```

The method would traverse up until the tail node. Once it reaches the tail node, however, the while loop would end and therefore not print the value of the tail node. We could update the method to check if the current node is equal to the tail, then break out of the loop. This way, the traversal would touch all of the nodes.

```
    void printList() {
        Node* current = head;

        while (current != nullptr) {
            std::cout << current->data << " ";
            current = current->next;

            if (current == tail) {
                std::cout << std::endl;
                break;
            }
        }
    }
```

However, the code directly above only works for circular lists with two pointers per node. If we go back to the original version of `printList`, this traversal will work with singly linked and doubly linked circular lists.

Insertion

Adding Nodes

At the Beginning

When adding a node to the beginning of a circular linked list, start by creating a new node. If the list is empty, the new node becomes both the `head` and `tail`, with its `next` attribute pointing back to itself. If the list is not empty, set the new node's next attribute to the current `head`, update the head pointer to point to the new node, and then update the `next` attribute of the `tail` to point to the new `head`.

The last step is to ensure that the next field of the tail node points to the new head. However, we need to consider the case where the newly added node is the only node in the list. To handle this scenario, we check if the head is `nullptr`, indicating that there is only one node in the list. If so, we set both the head and tail to the new node and have its `next` attribute point back to itself. Otherwise, we update the `next` attribute of the tail to point to the new `head`.

```
     void insertAtBeginning(int data) {
        Node* newNode = new Node(data);
        if (head == nullptr) {
            // List is empty, make the new node the head and
        tail.
            head = newNode;
            tail = newNode;
            newNode->next = newNode;
        } else {
            newNode->next = head;
            head = newNode;
            tail->next = newNode;
        }
    }
```

Create the `main` method. Use the `insertAtBeginning` method to add the values `1`, `2`, and `3` to the list. Then print the list.

```
int main() {
    CircularList list;
    list.insertAtBeginning(3);
    list.insertAtBeginning(2);
    list.insertAtBeginning(1);
    list.printList();
    return 0;
}
```

At the End

Start by creating a new node. Then check to see if the list is empty (`head == nullptr`). If so, set both the head and the tail to the new node. Set the `next` attribute of the new node to point back to itself. Use the return keyword to exit the method.

```
    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if(head == nullptr) {
            // List is empty, make the new node the head and
        tail.
            head = newNode;
            tail = newNode;
            newNode->next = newNode;
            return;
        }

    }
```

If the list is not empty, then set the `next` attribute to loop back to the head. Then have the `next` attribute of the old tail point to the new tail. Finally, set the `tail` attribute to the new node.

```
    void insertAtEnd(int data) {
        Node* newNode = new Node(data);

        if(head == nullptr) {
            // List is empty, make the new node the head and
        tail.
            head = newNode;
            tail = newNode;
            newNode->next = newNode;
            return;
        }

        newNode->next = head;
        tail->next = newNode; // Point to itself for
        circularity.
        tail = newNode;
    }
```

Update the `main` method so that it uses the `insertAtEnd` method to add the values `1`, `2`, and `3` to the list. Then print the list.

```
int main() {
    CircularList list;
    list.insertAtEnd(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.printList();
    return 0;
}
```

At a Specific Position

The `insertAtPos` method takes values for the data of the new node and the position for the insertion. Check to see if the position for insertion is less than 1. If so, print a message to the user about the invalid position, and exit the method with a `return` statement. Next, check to see if the position is 1. If so, call the `insertAtBeginning` method.

```
   void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        }

    }
```

Add an else statement to the conditional and create the `current` node. We also need a new node and `currentPos`, which is a counter for the current position. As long as `currentPos` is less than the desired insertion position minus 1, set `current` to the next node and increment `currentPos` by 1. Check to see if we have traversed the list (`current == head`). If so, print a message to the user that the position they provided was not valid (i.e., the position goes beyond the length of the list). Exit out of the method with a return statement.

```
    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before the
        desired position.
            while (currentPos < pos - 1) {
                current = current->next;
                currentPos++;

                if (current == head) {
                    std::cout << "Invalid position." <<
        std::endl;
                    return;
                }
            }

        }
    }
```

The last part is to insert the node into the position. Set the `next` attribute of the new node to the `next` attribute of the current node. Then set the `next` attribute of the current node to the new node. If the new node points to the head, set the tail to the new node.

```
    void insertAtPos(int data, int pos) {
        if (pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if (pos == 1) {
            insertAtBeginning(data);
        } else {
            Node* current = head;
            Node* newNode = new Node(data);
            int currentPos = 1;

            // Traverse the list to find the node before the
        desired position.
            while (currentPos < pos - 1) {
                current = current->next;
                currentPos++;

                if (current == head) {
                    std::cout << "Invalid position." <<
        std::endl;
                    return;
                }
            }

            // Adjust links to insert the new node at the
        specified position.
            newNode->next = current->next;
            current->next = newNode;
            if (newNode->next == head) {
              tail = newNode;
            }
        }
    }
```

Update the `main` method so that it uses the `insertAtEnd` method to add the values `1`, `2`, `3`, and `4` to the list. Use the `insertAtPos` method to insert `5` at position `3`. Then print the list.

```
int main() {
    CircularList list;
    list.insertAtEnd(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    std::cout << "Before: ";
    list.printList();

    std::cout << "After: ";
    list.insertAtPos(5, 3);
    list.printList();
    return 0;
}
```

challenge

Try these variations:

- Test the `insertAtPos` method with the valid positions: `1`, `2`, `3`, `4`, and `5`.
- Test the `insertAtPos` method with the invalid positions: `0` and `6`.

Deletion

Removing Nodes

From the Beginning

Deleting from the beginning has three different cases. If the list is empty (`head == nullptr`), then do nothing and use a `return` to exit the method. If the list only has one node, set both the head and tail to `nullptr` and exit the method with `return`. In all other cases, make the head point to the second node (`head->next`). Then have the tail point to the new head, and update the `prev` pointer of the new head to the tail.

```
    void deleteAtBeginning() {
        if (head == nullptr) return;

        if (head == tail) {
          head = nullptr;
          tail = nullptr;
          return;
        }

        head = head->next;
        tail->next = head;
    }
```

Update the `main` method so that the list contains the values `1`, `2`, `3`, and `4`. Then remove the first node from the list with the `deleteAtBeginning` method.

```
int main() {
    CircularList list;
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    std::cout << "Before: ";
    list.printList();

    std::cout << "After: ";
    list.deleteAtBeginning();
    list.printList();
    return 0;
}
```

From the End

Just as before, we are going to do nothing if the list is empty. If the list only has one node, then set both the head and tail to `nullptr` and exit the method. Things get a little more complicated in all other cases. We may know the tail of the list, but we cannot easily access it since our list is singly linked. Create a new node called `secondToLast` and set it to the head. Traverse the list until `secondToLast` points to the tail. Then have `secondToLast` point to the head and make `secondToLast` the tail.

```
    void deleteAtEnd() {
        if (head == nullptr) return;

        if (head == tail) {
            head = nullptr;
            tail = nullptr;
            return;
        }

        Node* secondToLast = head;
        while (secondToLast->next != tail) {
            secondToLast = secondToLast->next;
        }

        secondToLast->next = head;
        tail = secondToLast;
    }
```

Update the `main` method so that the list contains the values `1`, `2`, `3`, and `4`. Then remove the tail node from the list with the `deleteAtEnd` method.

```
int main() {
    CircularList list;
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    std::cout << "Before: ";
    list.printList();

    std::cout << "After: ";
    list.deleteAtEnd();
    list.printList();
    return 0;
}
```

From a Specific Position

Start by checking for three special cases. If the list is empty, print a message that you cannot remove a node from an empty list and exit the method. If the position is less than `1`, print a message that the position is invalid and exit the method. If the position is `1`, call the `deleteAtBeginning` method.

```
    void deleteAtPos(int pos) {
        if (head == null) {
          std::cout << "Cannot remove node from an empty list.";
          return;
        }

      if(pos < 1) {
        std::cout << "Invalid position.";
        return;
      }

      if(pos == 1) {
        deleteAtBeginning();
      }

    }
```

Add an else statement to the last conditional. Create the node `current` and set it to the head. Create `currentPos` which is a counter for the current position. As long as `currentPos` is less than the position for deletion minus 1, have `current` move to the next node in the list. Then increment `currentPos` by 1. If `current` is equal to the tail, the position to delete exceeds the length of the list. Print a message to the user. After the loop ends, set the value of `current->next` to `current->next->next`. This skips over the node that is to be removed from the list. Finally, check to see if `current->next` is pointing to the head. If so, set the tail to `current`.

```
    void deleteAtPos(int pos) {
        if (head == nullptr) {
          std::cout << "Cannot remove node from an empty list."
        << std::endl;
          return;
        }

        if(pos < 1) {
            std::cout << "Invalid position." << std::endl;
            return;
        }

        if(pos == 1) {
            deleteAtBeginning();
        } else {
            Node* current = head;
            int currentPos = 1;

            // Traverse the list to find the node before the
        desired position.
            while (currentPos < pos - 1) {
                current = current->next;
                currentPos++;

                if (current == tail) {
                    std::cout << "Invalid position." <<
        std::endl;
                    return;
                }
            }

            // Adjust links to insert the new node at the
        specified position.
            current->next = current->next->next;
            if (current->next == head) {
                tail = current;
            }
        }
    }
```

Update the `main` method so that the list contains the values `1`, `2`, `3`, and `4`. Then remove the node at position `2` with the `deleteAtPos` method.

```
int main() {
    CircularList list;
    list.insertAtBeginning(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    std::cout << "Before: ";
    list.printList();

    std::cout << "After: ";
    list.deleteAtPos(2);
    list.printList();
    return 0;
}
```

challenge

Try these variations:

- Test the `deleteAtPos` method with the valid positions: `1`, `2`, `3`, and `4`.
- Test the `deleteAtPos` method with the invalid positions: `0` and `5`.

Traversal & Searching

Navigating a Circular Linked List

Traversing

We have already seen how to traverse a list with the `printList` method. However, I would like to return to the one of the biggest challenges when working with circular lists, an infinite loop. Find the `printList` method in the IDE and comment out the `break` statement. Running this code will produce a lot (I mean a lot) of output before finally timing out. Pay particular attention when traversing a circular list. You have to manually stop the loop yourself.

```
    void printList() {
        Node* current = head;

        while(current != nullptr) {
          std::cout << current->data << " ";
          current = current->next;

          if(current == head) {
            std::cout << std::endl;
            // break;
          }
        }
    }
```

info

Important

Be sure to uncomment the `break` statement in the `printList` method. We do not want any more infinite loops when we run our code.

▼ **Traversing a circular list in reverse**

If you have a circular list that is doubly linked, you can traverse it in reverse. Instead of setting the `current` node to the head, set it to the tail. You will create an infinite loop just as before. Set `current` to `current->prev` to move backwards through the list. When `current` is equal to the tail, break out of the loop.

Important, the code below is an example of how to traverse a circular list in reverse. However, do not copy and paste it into the IDE. The code snippet assumes both `prev` and `next` pointers. Our code only has the `next` pointer.

```
    void reversePrint() {
        Node* current = tail;

        while(current != nullptr) {
          std::cout << current->data << " ";
          current = current->prev;

          if(current == tail) {
            std::cout << std::endl;
            break;
          }
        }
    }
```

Searching

To search for a specific node, create the `findNode` method that takes an integer value. The node with this value will be returned. Create the node `current` and assign it the value of `head`, and traverse the list. If the value of the current node is equal to the target value, return the current node. Advance to the next node in the list. If you get back to the head, break out of the loop. Breaking out of the loop means the node is not in the list, so return `nullptr`.

```
    Node* findNode(int targetValue) {
        Node* current = head;

        while(current != nullptr) {
          if (current->data == targetValue) {
            return current;
          }
          current = current->next;

          if(current == head) {
            break;
          }
        }

        return nullptr;
    }
```

Now that we can find a specific node, we need to be able to delete it. Create the `deleteNode`method that takes a `Node` object. Start by checking for special cases. If the list is empty (`head` is `nullptr`) or the node does not exist in the list (`del` is `nullptr`), do nothing and exit the method. If you are removing the first node or there only one node in the list (`head` equals `tail`) call the `deleteAtBeginning` method. This method handles both of those cases. If you are deleting the tail node, call the `deleteAtEnd` method.

```
    void deleteNode(Node* del) {
        if (head == nullptr || del == nullptr) return;

        if (head == del || head == tail) {
            deleteAtBeginning();
            return;
        }

        if (tail == del) {
            deleteAtEnd();
            return;
        }

    }
```

If the node to delete is elsewhere in the list, we are going to traverse the list, stopping at the node before the one to be deleted (`del`). Create the node `prevDel` and set it to the head. Traverse the list with this node. If the node after `prevDel` is the node to be deleted, break out of the loop. Otherwise advance to the next node. After leaving the loop, have the node before `del` point to the node after `del`.

```
    void deleteNode(Node* del) {
        if (head == nullptr || del == nullptr) return;

        if (head == del || head == tail) {
            deleteAtBeginning();
            return;
        }

        if (tail == del) {
            deleteAtEnd();
            return;
        }

        Node* prevDel = head;

        while (prevDel != nullptr) {
          if (prevDel->next == del) {
              break;
          }
          prevDel = prevDel->next;
        }

        prevDel->next = del->next;
    }
```

▼ Checking for the head

You may have noticed that this traversal did not check to see if `prevDel` has gone back to the head of the list. Normally, you would do this to avoid an infinite loop. If the node is not in the list, it will continue to traverse

without stopping. However, we know in advance if the node exists or not – if it has the value `nullptr` it is not in the list. We handled this case in the first conditional of the method.

Update the `main` method so the list has four nodes. Find and delete the node with the value of `3`. Then print the list.

```
int main() {
    CircularList list;
    list.insertAtEnd(1);
    list.insertAtEnd(2);
    list.insertAtEnd(3);
    list.insertAtEnd(4);
    std::cout << "Before: ";
    list.printList();

    std::cout << "After: ";
    Node* target = list.findNode(3);
    list.deleteNode(target);
    list.printList();
    return 0;
}
```

challenge

Try these variations:

- Test the `deleteNode` method with the valid values: `1`, `2`, `3`, and `4`.
- Test the `deleteNode` method with the invalid values: `0` and `5`.

Analyzing Circular Linked Lists

Time Complexity Analysis of Operations in Circular Linked Lists

Insertion

- **At the Beginning** - The time complexity for this operation is $O(1)$ because we only have to change a few pointers, assuming we maintain both a `head` and `tail` pointer.
- **At the End** - This operation has a time complexity of $O(n)$. Even if you know the `tail` pointer, a singly linked list means you have to traverse up to the node to delete. A doubly linked circular list can remove nodes at the end with a time complexity of $O(1)$.
- **At a Specific Position** - In the worst-case scenario, this could take $O(n)$ time, as we may have to traverse the entire list to reach the specific position.

Deletion

- **From the Beginning** - Deletion from the beginning has a time complexity of $O(1)$ because it involves just a few pointer updates.
- **From the End** - The time complexity is $O(1)$ for deletion from the end if a `tail` pointer is maintained.
- **Specific Position** - Just like insertion, deletion at a specific position could go up to $O(n)$ in the worst-case scenario.

Traversal & Searching

- **Traversal** - Traversing the entire list would take $O(n)$ time. Here, one should be careful to stop the traversal once we reach back to the head to avoid an infinite loop.
- **Searching** - In a circular linked list, searching involves traversing through the list until the item is found or the loop comes back to the head. Therefore, the time complexity is $O(n)$.

Table of Operations

The table below summarizes the time complexities for the various operations covered. Any reductions to time complexity are due to knowing the tail node.

Operation	Singly Linked	Doubly Linked	Circular Linked
Insertion (Beginning)	$O(1)$	$O(1)$	$O(1)$
Insertion (End)	$O(n)$ or $O(1)$	$O(n)$ or $O(1)$	$O(1)$
Insertion (Position)	$O(n)$	$O(n)$	$O(n)$
Deletion (Beginning)	$O(1)$	$O(1)$	$O(1)$
Deletion (End)	$O(n)$	$O(n)$ or $O(1)$	$O(1)$
Deletion (Position)	$O(n)$	$O(n)$	$O(n)$
Traversal	$O(n)$	$O(n)$	$O(n)$
Searching	$O(n)$	$O(n)$	$O(n)$

While circular linked lists offer some advantages like the ease of performing rotation operations and efficient insertions and deletions when they use the tail pointer and double pointers. However, searching and specific insertions or deletions can require traversing the list, making those operations $O(n)$ in the worst case.

Formative Assessment 1

Formative Assessment 2